GuessNumber Rebuild: Design Specification

Version 0.1 (draft for review) · 13 October 2026 · Prepared for Omoruyi Osemwegie
Sources: repository RuyiOne/GuessNumber (outer `GuessNumber/` project, `Scripts/GuessNumber.js`, `GuessHub.cs`); IJ-AI manuscript 28757 (user copy); results brief of the reviewer comments.

1. Purpose and scope

This document specifies a rebuild of GuessNumber as three clients (a React web application, a standalone mobile application and a Telegram/WhatsApp chatbot) served by a single ASP.NET Core game engine. It sets out the game rules, including the new score row and the number-change mechanism, the settings schema, three computer players, the platform architecture and an experiment harness. The harness is intended to reproduce every metric reported in manuscript 28757 and to address the principal second-round reviewer concerns. The document does not contain implementation code. Prompts for implementation in Cursor are provided separately in `CURSOR_HANDOFF.md`.

2. Findings from the existing code that constrain the design

The state, action, reward and network are taken from `Scripts/GuessNumber.js`, as the brief requires. Several properties of that code bear directly on the validity of the published comparison. They should be acknowledged rather than silently corrected.

Element	Repository behaviour	Consequence
Code space	All 4-digit codes with distinct digits 1 to 9: 3,024 codes (`generateUniqueNumbers`).	Consistent with the manuscript.
State	`reset()` returns a vector of 3,024 ones. Every `step()` returns a vector of 3,024 zeros.	The network receives no information about the game. In effect, Q-values are learned for two constant states only.
Action	An index into `environment.possibleNumbers`, which the environment filters after each guess to the codes consistent with all feedback. `actionSize` is reset to the length of the filtered list.	Both "DQN" agents already act only within the consistent candidate set, so they are close to sampling without replacement. The meaning of an action index shifts as the list shrinks. This probably explains the 100% success within 100 episodes (reviewer concerns B2.1 and C2.1).
Reward	100 on a solve, otherwise `dead*10 + wounded*2`.	The intermediate reward is dense, not sparse, and equals the new score formula. "Sparse reward" in the title needs qualification or a sparse variant.
Network	Dense 128 (ReLU) → 128 (ReLU) → 3,024 linear outputs. Adam with learning rate 0.01, MSE loss, one epoch per batch.	The learning rate in the code (0.01) differs from Table 1 (0.001 and 0.0001).
Hyperparameters in code	gamma 0.09, epsilon 0.1, decay 0.95, buffer 100, batch 5. The constructor defaults are gamma 0.09, buffer 1,000 and batch 64.	These do not match Table 1 (gamma 0.99, buffer 100,000, batch 64/32). The configuration that produced the published figures must be identified.

Exploration	`act()` returns `floor(r * actionSize)` when `r ≤ epsilon`.	Random actions are confined to the first epsilon fraction of indices, so exploration is biased.
PER sampling	Priorities are stored as $(\|\delta\|+10^{-5})^{\alpha}$ and then raised again to β to form the sampling probabilities.	The effective exponent is αβ, and β is not annealed. Standard PER uses $p_i^{\alpha}/\Sigma p^{\alpha}$ and applies β only to the importance-sampling weights.
Scoring functions	`evaluateGuess` (environment) and `evaluateGuess2` (game).	Equivalent for codes with distinct digits. A single shared scorer is specified below.

The rebuild therefore runs two configurations side by side. The first is a **faithful port** (configuration R0), which replicates the repository behaviour so that the published curves can be regenerated. The second is a **corrected configuration** (configuration R1), which has an informative state, stable action indices, unbiased exploration and standard PER. Any difference between R0 and R1 is itself reportable.

3. Game rules

3.1 Codes, feedback and turns

- A secret is four distinct digits from 1 to 9. The default variant has 3,024 possible secrets.
- For a guess g against a secret s, *dead* = $|\{i : g_i=s_i\}|$ and *wounded* = $|\text{set}(g)\cap\text{set}(s)|$ − dead.
- Players alternate turns. The inviter, or the human in a game against the computer, moves first. On each turn a player makes either a guess or a number change (see 3.3).
- A player wins when a guess scores four dead. Scores are pre-wired for an equal-turns rule, under which the second player receives one final turn after the first player solves, but this rule is off by default (Section 10, item 4).

3.2 Score row

- Each guess earns **dead × 10 + wounded × 2** points for the player who made it. For example, 1 dead and 1 wounded earns 12, and a solving guess earns 40.
- A number change costs 25 points.
- A player's score is the sum of their guess points minus 25 for each change. It cannot fall below zero, because a change requires a score of at least 25.
- Score determines ranking within a finished game and enters the history and the Elo tie-break. Winning is determined by solving first, not by score.
- Guess table: rows are appended from top to bottom in turn order, one column per player. A **Score** row is pinned beneath the scrolling body (in a `tfoot`, with sticky positioning on mobile) and shows each player's running total under that player's column. A change appears as a row annotated “−25”.

3.3 Number-change mechanism

Any player, whether human or computer, may replace **exactly two** digit positions of **their own** secret. The change uses that player's turn and costs 25 points.

#	Condition	Rule
E1	Score below 25	Rejected with `INSUFFICIENT_SCORE`. The turn is not consumed.

E2	Allowance used up (once-per-game mode)	Rejected with `CHANGE_LIMIT_REACHED`.
E3	Resulting code repeats a digit, or contains 0	Rejected with `INVALID_CODE`. A new digit may not duplicate one of the two retained digits, but two selected positions may swap their digits with each other.
E4	Fewer or more than two positions differ	Rejected. A swap of two positions counts as two changed positions.
E5	New code equals a code the opponent has already guessed	Allowed. The opponent's earlier feedback is history and is not re-evaluated.
E6	New code equals the player's previous secret from any earlier change	Allowed, because no rule forbids it. It is logged for analysis.
E7	The opponent's most recent guess solved the code	Not possible, because the game has already ended.
E8	Equal-turns rule active and the opponent has solved	Changes are disabled during the final turn.
E9	Concurrency	The server applies commands in sequence for each game. A change and a guess cannot both be applied in one turn.
E10	Disconnect during the change dialog	Nothing is committed until the server confirms. The turn timer continues to run.

Notification setting. With `notifyOnChange = true`, the opponent receives `OpponentChangedSecret{turn}`, which states that a change occurred and on which turn, but not which positions changed. With `false`, nothing is sent. Secrets are never sent to any client. The game record stores the full history for replay after the game ends.

4. Settings schema

```
GameSettings {
  ruleset:        "classic4"            // or "classic5", "pool0to9", "repeatsAllowed" (experiment variants)
  numberChange:   "once" | "unlimited" | "disabled"     // default "once"
  changeCost:     25                    // fixed in ruleset; not user-editable in casual play
  changeDigits:   2                     // fixed
  notifyOnChange: true                  // opponent informed that a change occurred
  changeConsumesTurn: true
  turnTimerSec:   60 | 30 | 0
  equalTurns:     false
  opponent:       { kind: "human" } | { kind: "computer",
                    agent: "swor" | "dqn" | "double" | "duelling" | "doubleDuelling",
                    per: true | false,        // false = uniform replay without replacement; ignored for "swor"
                    allowComputerChange: true, changePolicy: "never" | "heuristic" }
}
PlayerSettings { displayName, locale ("en-GB"), theme, defaultSolver, notifications }
```

Settings are fixed when a game is created and are stored with the game, so replays and experiments remain reproducible. The Settings tab edits the defaults for new games.

5. Computer players

The user requirement for Computers A, B and C has been generalised. The player selects an **agent type** (sampling without replacement, DQN, Double DQN, Duelling DQN or Double+Duelling) and, for the four DQN types, toggles **PER**. PER on corresponds to the original Computer B, and PER off (uniform replay without replacement) to Computer C. Rainbow is out of scope for this rebuild. All solvers implement a single interface, which the game engine and the experiment harness both call:

```
interface ISolver { Reset(ruleset, seed); string NextGuess(); Observe(guess, dead, wounded);
                    OnOpponentChange(turn /* or null if unannounced */); SolverStats Stats(); }
```

5.1 Computer A: sampling without replacement

This is a port of `getUniqueNumbers`, `makeAIGuess` and `filterAIPossibleNumbers`.

```
Reset: C ← all valid codes; H ← []          // H = history of (guess, feedback)
NextGuess: g ~ Uniform(C \ guessed); guessed ← guessed ∪ {g}; return g
Observe(g,f): H.append(g,f); C ← {c ∈ C : score(g,c) = f}
```

5.2 DQN family (DQN, Double, Duelling, Double+Duelling) with PER on/off

This is a port of `DQN`, `PrioritizedReplayBuffer` and `DQNAgent`. Candidate pruning (sampling without replacement) restricts the legal actions.

```
State   s: R0 = constant vector (as repository); R1 = candidate mask m ∈ {0,1}^3024 (+ turn index, last feedback)
Action  a ∈ {0..3023}, a fixed index into the full code list; illegal actions masked to −∞ (R1)
        (R0 reproduces the shifting index into the filtered list)
Reward  r = 100 if dead = 4, else dead*10 + wounded*2           // repository; sparse variant in §7
Net     Dense128-ReLU → Dense128-ReLU → Linear(3024); Adam; MSE weighted by IS weights
loop each step:
   a ← ε-greedy over legal actions (uniform over legal set, R1)
   observe r, s'; store (s,a,r,s',done) with priority p = (|δ|+ε0)^α
   sample batch i ~ p_i^α / Σ p^α    (R1; R0 reproduces repository exponent)
   w_i = (N·P(i))^(−β) / max w;  β annealed β0 → 1
   target:  DQN, Duelling:        y = r + γ(1−d)·max_{a' legal} Q_θ−(s',a')
            Double, Double+Duel:  a* = argmax_{a' legal} Q_θ(s',a');  y = r + γ(1−d)·Q_θ−(s',a*)
   network: DQN, Double:          shared trunk → Linear(3024)
            Duelling variants:    trunk → V(s) [Dense64→1] and A(s,a) [Dense64→3024];
                                  Q = V + A − mean_{legal} A
   PER on:  update priorities with |y − Q(s,a)|;  PER off: see 5.3
   every K steps (or each episode, R0): θ− ← θ
```

5.3 PER off: uniform replay without replacement

With PER off, each DQN variant uses the same state, action, reward, network and candidate masking as its PER-on counterpart. Minibatches are drawn uniformly **without replacement** from the buffer (no index appears twice within a batch), with no priorities and no importance-sampling weights. The PER toggle is therefore a clean ablation for each variant.

5.4 Handling a number change (all solvers)

Let H be the feedback history and t_c the turn of the change. $N_2(c)$ denotes the set of valid codes that differ from c in exactly two positions.

```
OnOpponentChange(t_c) [announced]:
    C_pre ← codes consistent with H[< t_c]
    C     ← ∪_{c ∈ C_pre} N2(c)  filtered by consistency with H[≥ t_c]
Unannounced (notifyOnChange = false): detect when C becomes empty, then
    for t in latest..earliest candidate change turns: try the announced rule with t_c = t;
    accept the first non-empty C; if none, C ← codes consistent with H[last k] only (k=2)
```

- **Computer A** uses the rule above directly. The set of already-guessed codes is cleared, so that a code ruled out earlier may be guessed again.

- **All DQN variants** recompute the candidate mask by the same rule. In R1 the state is the mask, so the network observes the change through its input. Transitions stored before t_c are retained but tagged `stale`. With PER on, stale transitions receive maximum priority once, so that the effect of PER on non-stationarity can be measured. Epsilon is temporarily raised to $\max(\varepsilon, \varepsilon_{change}=0.2)$ and then decays again.
- **Computer's own change policy** (`heuristic`): change when the opponent's best feedback is 3 dead, or when the opponent's estimated candidate set size is 6 or fewer, provided score ≥ 25 and allowance remains. The two positions are chosen to maximise the expected size of the opponent's consistent set. This is offered as an option, not as a learned policy.

6. Platform architecture

Layer	Specification
Game engine	`GuessNumber.Engine`, a .NET 9 class library with no I/O: rules, scorer, validator, the number-change protocol and the solvers (sampling without replacement and the four DQN variants, with inference through TorchSharp or ONNX Runtime). It is deterministic given a seed.
Server	ASP.NET Core 9: `GameHub` (SignalR, strongly typed), REST endpoints for profiles, history and settings, ASP.NET Core Identity with JWT, PostgreSQL through EF Core, and Redis for presence, the queue and the SignalR backplane. The server is authoritative: secrets are stored encrypted and never sent to clients.
Web	React 19, TypeScript, Vite and Bootstrap 5 (React-Bootstrap), with `@microsoft/signalr`, TanStack Query and Zustand. The screens are Game, Lobby, Settings and History.
Mobile	React Native (Expo) sharing the TypeScript domain types and the hub client package with the web client. It works as a standalone app: offline games against Computer A use a TypeScript port of the scorer and solver, and online play uses the hub.
Chatbot	`GuessNumber.Bot` worker: a Telegram Bot API webhook and a WhatsApp Cloud API webhook, both mapped to the same engine commands (`/new A`, `/guess 1234`, `/change 1:4 3:1`, `/score`). The table is rendered as monospaced text, with the score line last.
Training and experiments	A Python 3.12 package `gn_rl` (PyTorch and Gymnasium) wrapping an environment that is bit-identical to the engine rules. Parity is checked against the C# scorer on a fixed test vector. Trained policies are exported to ONNX for use in the game.

Python is proposed for training because the reviewers' baselines (Double and Duelling DQN) and the statistical tooling (rliable, SciPy) are mature there. The TF.js port is retained only to replicate configuration R0.

7. Experiment plan

7.1 Reproduction of the published metrics

For R0, with DQN+PER and DQN with uniform replay as in the manuscript, run 10 seeds × 1,000 episodes using the Table 1 hyperparameters. Log the following for each episode: total reward, success, cumulative reward, guesses, loss for each update, TD errors, priorities, IS weights and ε. Regenerate Figures 3 to 7 (learning curve, success rate, cumulative reward, sample efficiency and loss distribution) at 300 dpi or higher.

7.2 Mapping to reviewer concerns

Concern	Experiment or output
B2.1, C2.1, B2.6, C2.6: task too easy; need larger spaces and more delayed reward	Variants: `classic5` (5 digits, 15,120 codes) and `pool0to9` (4 of 10 digits, 5,040 codes). Reward modes: *shaped* (repository) and *sparse* (+100 on a solve only, 0 otherwise). Settings: candidate masking on and off, and number change at a random turn in [3, 6]. Episode cap of 15 guesses.
B2.2, C2.2: independently tuned hyperparameters	(i) Primary: one shared hyperparameter set for all eight agents (γ 0.99, learning rate 0.0005, batch 64, buffer 100,000, ε 1.0 → 0.01 at decay 0.995, target sync every 500 steps, α 0.6, β_0 0.4 → 1.0). The value of the learning rate is proposed and is to be fixed before the runs. (ii) Secondary sensitivity check: an equal tuning budget of 30 Optuna trials for each agent on validation seeds 100 to 104, with the final runs on seeds 0 to 9.
B2.3, C2.5, G1.2: statistics	Mean ± 95% bootstrap CI over 10 seeds (and 30 seeds for the headline comparison). IQM and performance profiles (rliable). Welch t-tests and Mann–Whitney U tests with Holm correction. Effect sizes: Cohen's d and Cliff's δ.
B2.4, C2.3: baselines	Core matrix: {DQN, Double DQN, Duelling DQN, Double+Duelling} × {PER on, PER off} = 8 agents, with **one shared hyperparameter set** and the same 10 seeds × 1,000 episodes. Non-learning references: random legal policy and sampling without replacement. Rainbow is out of scope. Its omission is stated as a limitation, and Double+Duelling+PER is the nearest combination evaluated.
B2.5, G1.3: separating metrics	Guesses to solve on 500 held-out secrets (mean, median and distribution); episodes to 95% success; area under the learning curve; success under a guess cap of 7 and of 10.
Robustness under number change	Success rate and extra guesses after a change, with notification on and off; recovery time in guesses; cumulative stale-transition sampling fraction (PER-on agents only).
C2.4, G1.5: interpretation of the PER loss	TD-error and priority distributions over training; IS-weight statistics; fraction of sampled terminal and stale transitions; loss shown next to evaluation reward. Repeat with learning rate and batch size equalised to test whether the elevated late-training loss is caused by the configuration.
PER ablation	$\alpha \in \{0, 0.4, 0.6, 1.0\} \times \beta_0 \in \{0.4, 0.5, 1.0\}$, with and without annealing; buffer size $\in \{10^3, 10^4, 10^5\}$.
G1.4: computational cost	Wall-clock time, environment steps per second and updates per second for each agent, with the hardware stated.
Head-to-head	1,000 games of each solver against Computer A, recording win rate with a 95% Wilson interval.

The harness is configured from a single YAML grid, writes Parquet logs, and produces figures and LaTeX tables from a single command. Seeds govern the network initialisation, the secret sequence and the exploration noise.

8. Phased build plan

Phase	Content	Exit criterion
0	Repository hygiene: a new monorepo (`engine/`, `server/`, `web/`, `mobile/`, `bot/`, `rl/`), with CI, and the legacy project archived.	CI passes on an empty skeleton.
1	Engine: rules, scorer, validator, number-change protocol and Computer A, with property-based tests.	100% of the rule tests (E1 to E10) pass.
2	Server: hub, persistence, authentication and settings.	A complete game against Computer A via an integration test.

3	Web UI: the game screen with the pinned score row, settings and the change dialog.	Playwright end-to-end tests of a human-vs-human game and a change.
4	RL package: R0 port (parity with TF.js) and R1; the four DQN variants with the PER toggle; ONNX export.	R0 reproduces the published curves within the CI bands.
5	Experiment harness and the full grid; figures and tables for the revision.	All rows of the table in 7.2 produced.
6	Mobile application (Expo).	Offline game against Computer A; online game through the hub.
7	Telegram and WhatsApp bot.	A complete game in each channel against any agent type.

Phases 4 and 5 can proceed in parallel with phases 2 and 3. Given that the second-round revision is pending, it is suggested that phases 1, 4 and 5 are prioritised.

9. Mockups

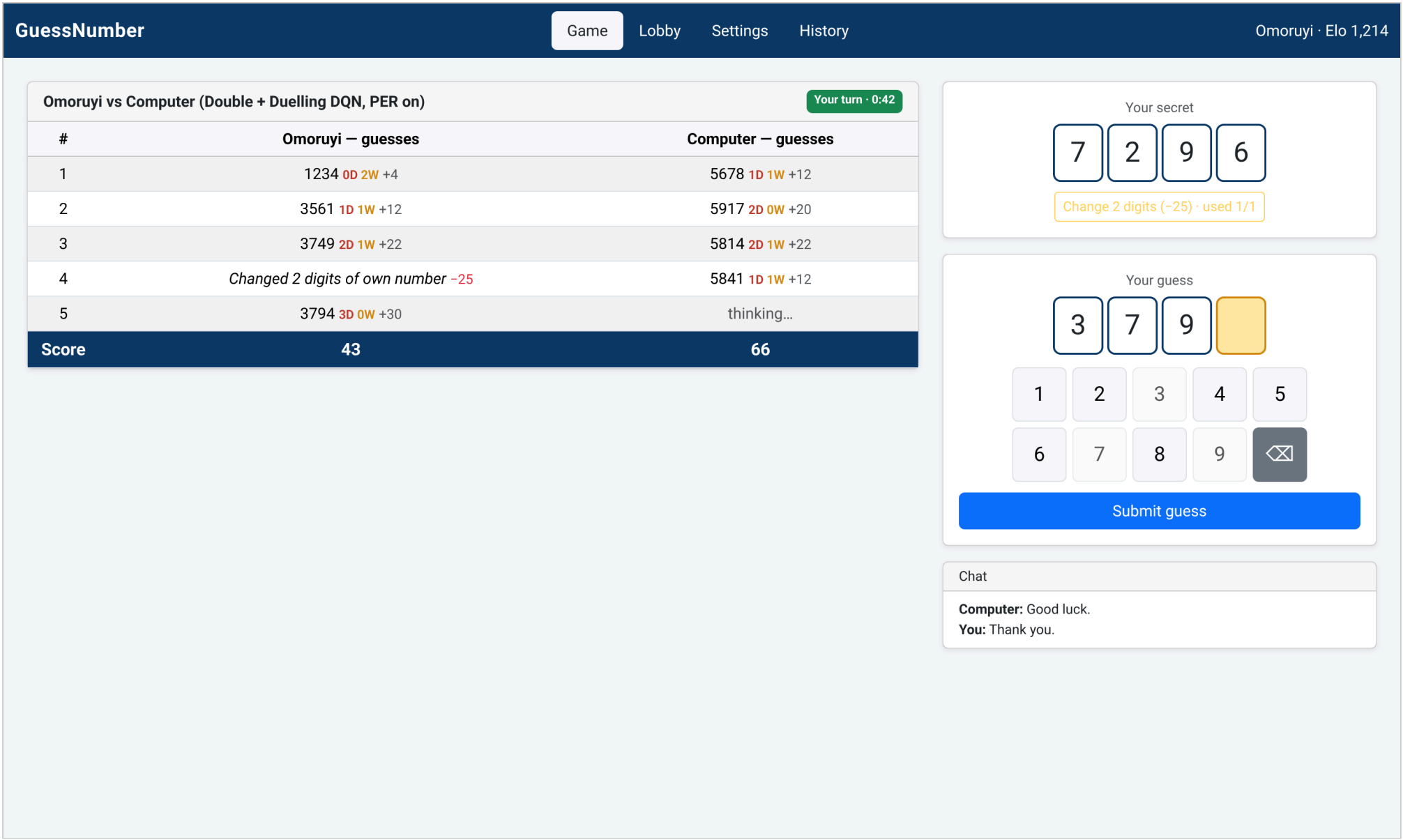


Figure 1. Game screen. Guess rows are appended from top to bottom, and the Score row is pinned beneath each player's column. The change row shows the 25-point deduction.

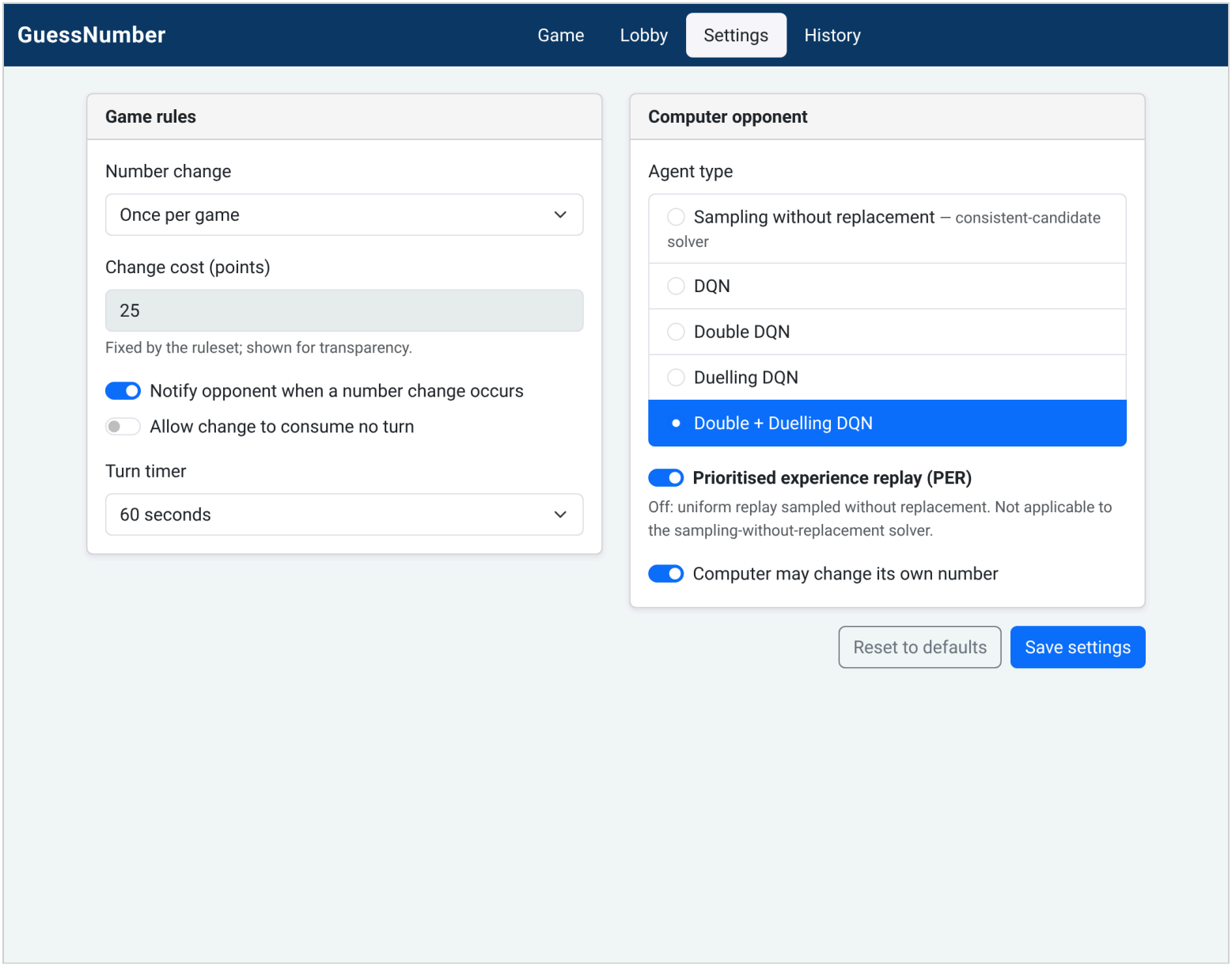


Figure 2. Settings tab: number-change mode, opponent notification, agent type and the PER toggle.

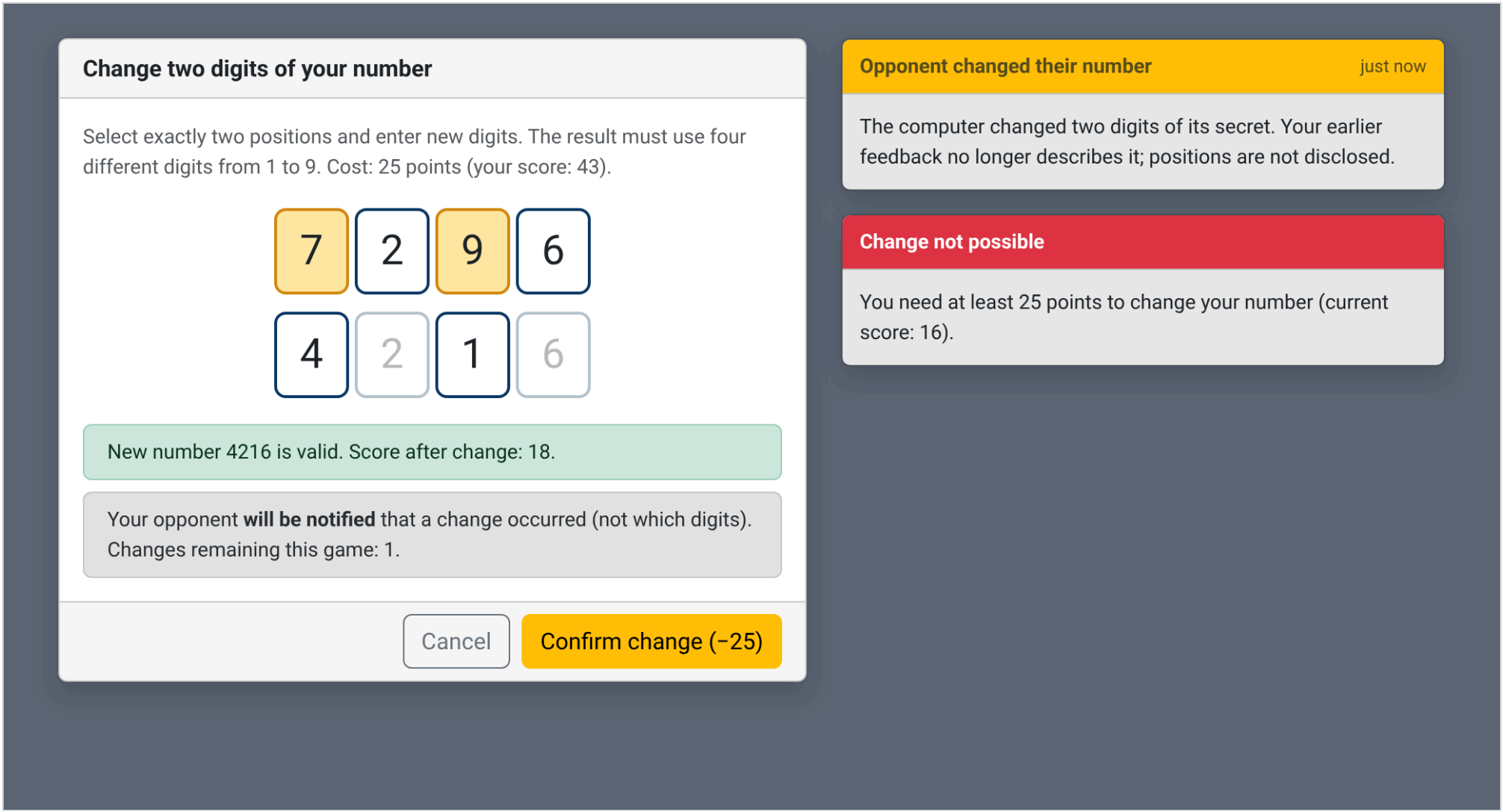


Figure 3. Number-change dialog with validation, the notification received by the opponent, and the rejection shown when the score is insufficient.

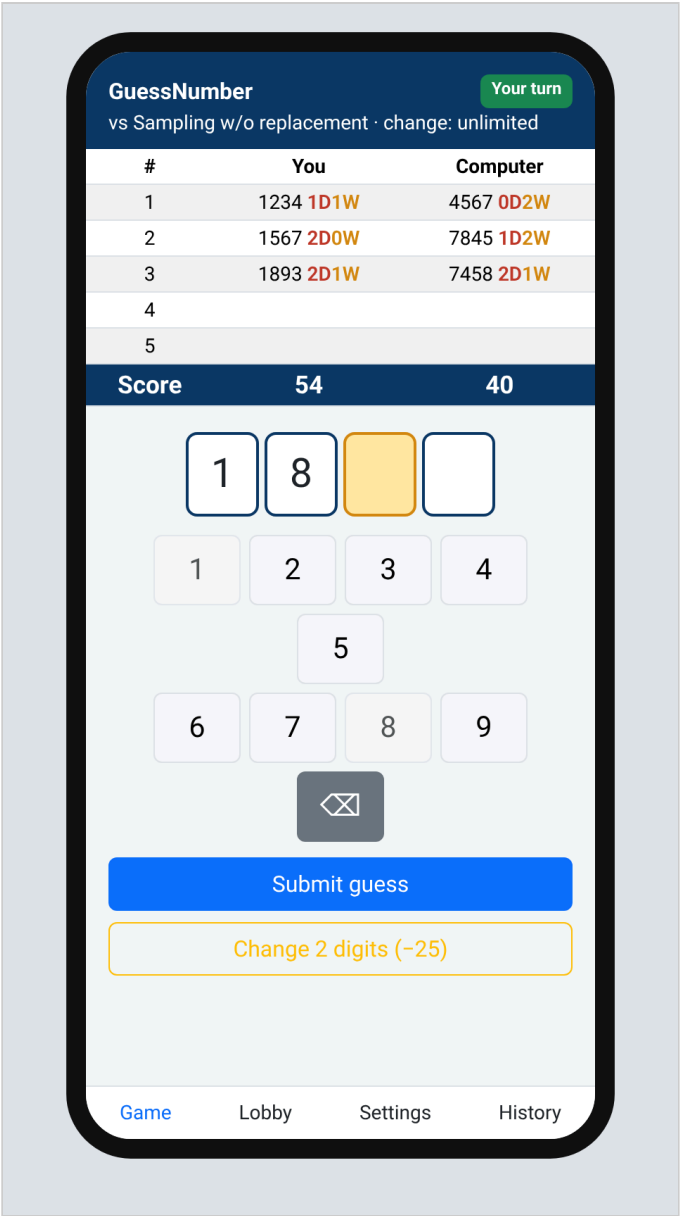


Figure 4. Mobile frame showing the same table, the pinned score row and the change action.

10. Open questions and assumptions

1. **"Sampling without replacement" with PER off.** This specification reads it as uniform minibatch sampling without replacement from the replay buffer. Candidate masking of illegal actions applies to all agents regardless of the toggle. Confirmation is requested.
2. **Shared hyperparameters.** The single set differs from both columns of Table 1, so the R0 replication uses Table 1 and the new matrix uses the shared set.
3. **"Change 2 digits".** Read as exactly two positions. A two-position swap is permitted.
4. **Whether a change consumes a turn.** Assumed yes, and made configurable.
5. **The equal-turns rule** was not specified, so it is off by default.
6. **Whether the solving guess earns 40 points or carries a bonus.** Assumed 40, with no bonus. The RL reward of 100 on a solve is kept separate from the display score.
7. **Which configuration produced the published figures.** The hyperparameters in the repository and in Table 1 disagree, so the actual configuration should be confirmed before the R0 replication is treated as authoritative.
8. **Notification content.** Assumed to state the turn of the change but not the positions.